

Chương ?? đã trình bày quá trình phân tích, thiết kế và xây dựng hệ thống IAM ở mức tổng thể, từ kiến trúc microservices hướng sự kiện cho đến thiết kế cơ sở dữ liệu và các luồng nghiệp vụ chính. Trên nền tảng đó, chương này tập trung làm rõ những đóng góp mang tính giải pháp của đề tài — tức là những quyết định thiết kế giải quyết một vấn đề cụ thể của bài toán phân quyền cho lập trình viên ngân hàng mà các nền tảng IAM thông dụng chưa đáp ứng trọn vẹn. Khác với Chương ?? nặng về mô tả *hệ thống được xây dựng như thế nào*, chương này phân tích sâu hơn về *tại sao* mỗi giải pháp được lựa chọn, cùng các đánh đổi (trade-off) đi kèm.

Bốn đóng góp được trình bày theo thứ tự từ mô hình dữ liệu đến vận hành và tích hợp. Mục 0.1 đề xuất việc mở rộng mô hình phân quyền dựa trên vai trò (RBAC) bằng chiều thứ hai là vị trí công tác. Mục 0.2 trình bày cơ chế quản trị vòng đời quyền một cách tự động, gắn trực tiếp với vòng đời nhân sự, bao gồm quy trình phê duyệt có kiểm soát và khả năng thu hồi tức thì. Mục 0.3 làm rõ cách mô hình hóa một quyền và nhúng vào JSON Web Token để thực thi kiểm soát truy cập phi trạng thái tại API Gateway. Cuối cùng, Mục 0.4 minh chứng tính khả dụng thực tế của nền tảng thông qua việc tích hợp một ứng dụng nội bộ hoàn chỉnh. Các cơ chế nền tảng kỹ thuật đi kèm như engine xác thực đa yếu tố (MFA) được thiết kế theo mô hình cắm thêm (plugin) qua giao diện *Authenticator*, song không phải trọng tâm phân quyền nên chỉ được nhắc đến ở mức cần thiết.

0.1 Mở rộng mô hình RBAC theo cặp Vai trò–Vị trí công tác

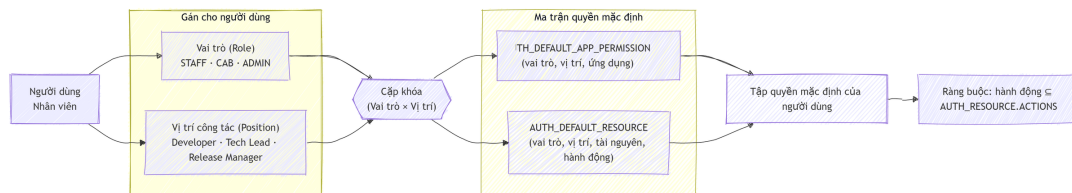
Đặt vấn đề

Mô hình kiểm soát truy cập dựa trên vai trò (Role-Based Access Control — RBAC) gán quyền cho vai trò và gán vai trò cho người dùng, nhờ đó đơn giản hóa việc quản trị so với việc cấp quyền trực tiếp cho từng cá nhân. Tuy nhiên, khi áp dụng vào môi trường phát triển phần mềm ngân hàng, mô hình RBAC thuần túy bộc lộ một hạn chế về độ mịn (granularity). Trong cùng một bộ phận, nhiều nhân sự cùng mang vai trò nghiệp vụ là *lập trình viên* nhưng lại đảm nhiệm các vị trí công tác khác nhau — lập trình viên (Developer), trưởng nhóm kỹ thuật (Tech Lead), quản lý phát hành (Release Manager) — và mỗi vị trí đòi hỏi một tập quyền mặc định khác nhau trên cùng một ứng dụng. Nếu chỉ dùng vai trò làm chiều phân quyền duy nhất, hệ thống buộc phải lựa chọn giữa hai phương án đều không tối ưu: hoặc tạo ra rất nhiều vai trò chi tiết (dẫn đến hiện tượng bùng nổ vai trò — *role explosion*, làm việc quản trị trở nên rối rắm), hoặc cấp một tập quyền chung rộng cho cả vai trò (vi phạm nguyên tắc đặc quyền tối thiểu — *least privilege*). Ở chiều ngược lại, mô hình kiểm soát truy cập theo thuộc tính (ABAC) tuy linh hoạt nhưng quyết định cấp quyền được tính động theo nhiều thuộc tính nên khó tĩnh-hóa, khó kiểm toán và khó giải thích — những yếu tố tối quan trọng trong môi trường chịu

kiểm soát chặt chẽ như ngân hàng.

Giải pháp đề xuất

Đề tài đề xuất mở rộng RBAC bằng cách bổ sung **vị trí công tác** làm chiều phân quyền thứ hai bên cạnh vai trò, hình thành mô hình ánh xạ (Vai trò $\times$ Vị trí) $\rightarrow$ Tập quyền. Quyền mặc định không được mã hóa cứng trong chương trình mà được khai báo dưới dạng dữ liệu trong hai bảng ma trận: AUTH_DEFAULT_APP_PERMISSION lưu quyền truy cập ở mức ứng dụng theo bộ ba (vai trò, vị trí, ứng dụng), và AUTH_DEFAULT_RESOURCE lưu quyền ở mức tài nguyên theo bộ bốn (vai trò, vị trí, tài nguyên, danh sách hành động). Mỗi ô của ma trận tương ứng với một bộ quyền mặc định và được ràng buộc bởi một khóa duy nhất, đảm bảo không tồn tại hai khai báo mâu thuẫn cho cùng một cặp vai trò–vị trí. Hình 0.1 minh họa cấu trúc của mô hình này.



Hình 0.1: Mô hình RBAC mở rộng theo cặp Vai trò $\times$ Vị trí công tác

Một điểm quan trọng trong thiết kế là cơ chế kiểm tra tính hợp lệ của hành động: khi một bộ quyền mặc định được khai báo cho một tài nguyên, hệ thống bắt buộc tập hành động yêu cầu phải là tập con của tập hành động mà tài nguyên đó định nghĩa (lưu trong trường ACTIONS của bảng AUTH_RESOURCE). Ràng buộc này ngăn chặn việc cấp những hành động không tồn tại trên tài nguyên, giữ cho ma trận quyền luôn nhất quán với mô hình tài nguyên–hành động đã định nghĩa ở mặt phẳng cấu hình.

Kết quả đạt được

Việc bổ sung chiều vị trí công tác cho phép biểu diễn các khác biệt quyền hạn tinh tế giữa các nhân sự cùng vai trò mà không làm bùng nổ số lượng vai trò, đồng thời vẫn giữ được tính tường minh và khả năng kiểm toán vốn là thế mạnh của RBAC. Do toàn bộ quyền mặc định được biểu diễn dưới dạng dữ liệu, quản trị viên có thể tra cứu, điều chỉnh và mở rộng ma trận quyền trực tiếp qua giao diện quản trị mà không cần can thiệp vào mã nguồn hay triển khai lại hệ thống. Quan trọng hơn, ma trận quyền mặc định này trở thành nguồn dữ liệu nền tảng cho cơ chế tự động hóa phân quyền theo vòng đời nhân sự được trình bày trong Mục 0.2.

0.2 Quản trị tự động vòng đời quyền theo vòng đời nhân sự

Đóng góp được trình bày trong mục này là phần cốt lõi của đề tài. Thay vì coi việc cấp quyền là một thao tác hành chính rời rạc do quản trị viên thực hiện thủ công, hệ thống gắn vòng đời của quyền truy cập trực tiếp vào vòng đời công tác của nhân sự, sao cho mỗi biến động nhân sự đều tự động kéo theo biến đổi quyền tương ứng. Mục này gồm ba phần: cơ chế tự động cấp, thu hồi và tạm treo quyền theo sự kiện nhân sự (0.2.1); quy trình phê duyệt có kiểm soát cho các quyền bổ sung ngoài mặc định (0.2.2); và cơ chế thu hồi quyền tức thì không phụ thuộc vào thời gian sống của token (0.2.3).

0.2.1 Tự động cấp, thu hồi và tạm treo quyền theo sự kiện nhân sự

Đặt vấn đề

Trong các nền tảng IAM thông dụng, việc cấp và thu hồi quyền khi nhân sự thay đổi trạng thái công tác phần lớn vẫn là thao tác thủ công. Cách làm này phát sinh ba rủi ro điển hình. Thứ nhất, khi tiếp nhận nhân viên mới, quản trị viên phải tự tay cấp từng quyền, vừa tốn thời gian vừa dễ cấp thiếu hoặc thừa. Thứ hai, khi nhân viên được luân chuyển sang vị trí mới, quyền của vị trí cũ thường không được thu hồi đồng thời, dẫn đến hiện tượng tích lũy đặc quyền (*privilege creep*) — người dùng dần sở hữu nhiều quyền hơn mức công việc hiện tại đòi hỏi. Thứ ba, khi nhân viên thôi việc, nếu quản trị viên quên thu hồi thì quyền truy cập vẫn tồn tại, tạo lỗ hổng bảo mật nghiêm trọng.

Giải pháp đề xuất

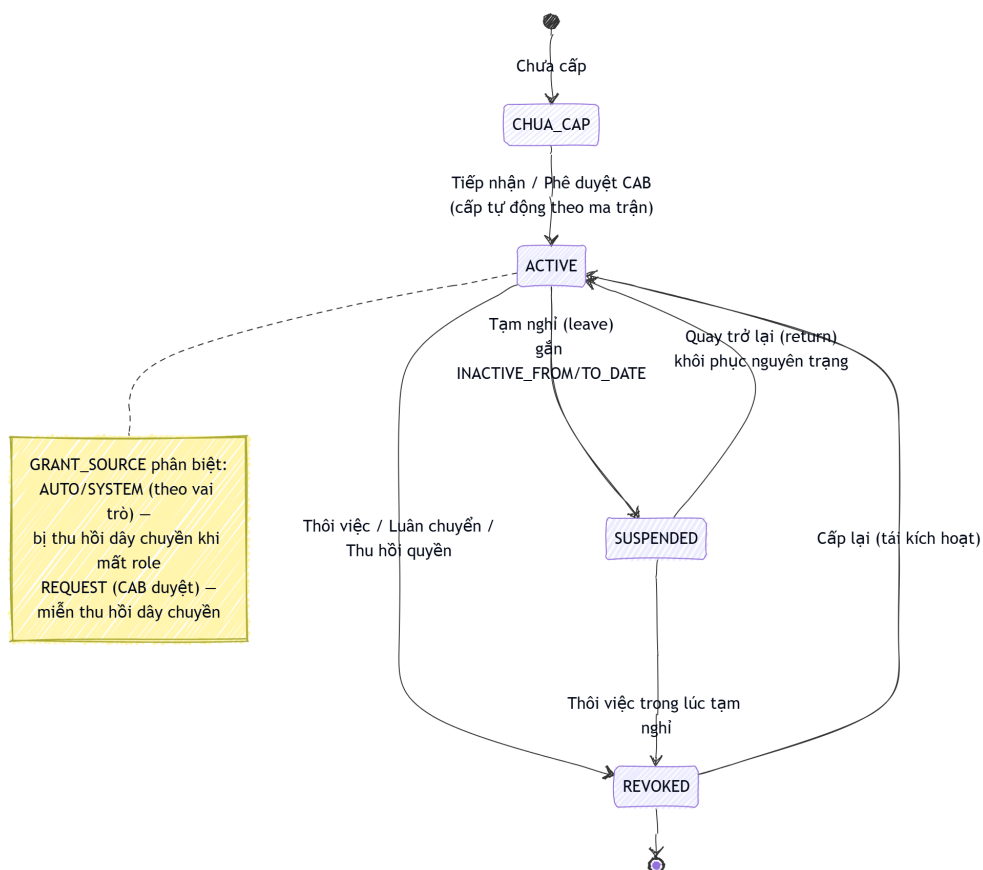
Hệ thống định nghĩa năm sự kiện vòng đời nhân sự — tiếp nhận (onboard), luân chuyển (transfer), tạm nghỉ (leave), quay trở lại (return) và thôi việc (offboard) — và gắn mỗi sự kiện với một phép biến đổi trạng thái quyền được thực thi tự động. Cơ chế tự động hóa được hiện thực thông qua kiến trúc hướng sự kiện: sau khi giao dịch cập nhật thông tin nhân sự được cam kết thành công, *iam-identity-service* phát một sự kiện Kafka tương ứng, và việc cấp quyền thực tế được thực hiện bất đồng bộ bởi bên tiêu thụ sự kiện.

Khi tiếp nhận hoặc luân chuyển, hệ thống phát sự kiện `DEFAULT-GRANT-PERMISSION-USER` mang theo danh sách vai trò và mã vị trí mới của nhân viên. Bên tiêu thụ *GrantPermissionConsumer* — nằm ngay trong chính *iam-identity-service* — tra cứu ma trận quyền mặc định đã trình bày ở Mục ?? theo cặp vai trò–vị trí, rồi cấp toàn bộ quyền tương ứng vào hai bảng `AUTH_APP_PERMISSION` và `AUTH_USER_RESOURCE` mà không cần bất kỳ lời gọi HTTP nào. Trong trường hợp luân chuyển, hệ thống thu hồi toàn bộ vai trò và quyền cũ trước khi phát sự kiện cấp quyền mới, nhờ đó loại bỏ triệt để hiện tượng tích lũy đặc quyền. Luồng

xử lý chi tiết của nghiệp vụ này đã được mô tả trong Hình ?? của Chương ??.

Đối với trạng thái tạm nghỉ dài hạn, hệ thống áp dụng một cách xử lý tinh tế hơn so với thu hồi–cấp lại thông thường. Thay vì xóa quyền, hệ thống chuyển quyền sang trạng thái SUSPENDED kèm theo mốc thời gian hiệu lực trong các trường INACTIVE_FROM_DATE và INACTIVE_TO_DATE. Khi nhân viên quay trở lại, quyền được khôi phục nguyên trạng bằng cách đưa trạng thái về ACTIVE và xóa các mốc thời gian, mà không phải tái tạo lại toàn bộ tập quyền từ ma trận mặc định. Cách làm này bảo toàn được cả những quyền bổ sung mà nhân viên đã được cấp riêng trước khi nghỉ. Hai luồng tạm nghỉ và quay trở lại đã được trình bày trong Hình ??.

Để phục vụ việc kiểm soát chính xác phạm vi thu hồi, mỗi bản ghi quyền được gắn một nhãn nguồn cấp (GRANT_SOURCE) phân biệt quyền cấp tự động theo ma trận mặc định và quyền cấp qua quy trình phê duyệt. Hình 0.2 biểu diễn máy trạng thái của một bản ghi quyền xuyên suốt các biến động vòng đời nhân sự.



Hình 0.2: Máy trạng thái của một bản ghi quyền theo vòng đời nhân sự

Kết quả đạt được

Cơ chế này loại bỏ hoàn toàn thao tác cấp quyền mặc định thủ công: việc tiếp nhận một nhân viên mới chỉ còn đòi hỏi quản trị viên nhập hồ sơ và gán vai trò

cùng vị trí, toàn bộ quyền mặc định được cấp tự động trong vài giây. Việc luân chuyển đảm bảo quyền cũ được thu hồi đồng thời với việc cấp quyền mới, triệt tiêu rủi ro tích lũy đặc quyền. Cách xử lý tạm treo theo mốc thời gian giúp phản ánh chính xác trạng thái công tác tạm thời mà vẫn bảo toàn được lịch sử quyền của nhân viên.

0.2.2 Quy trình phê duyệt có kiểm soát và tách biệt nguồn cấp quyền

Đặt vấn đề

Tập quyền mặc định theo vai trò và vị trí chỉ đáp ứng các nhu cầu công việc cơ bản. Trên thực tế, lập trình viên thường xuyên cần các quyền bổ sung mang tính tình huống — chẳng hạn truy cập một kho mã nguồn đặc thù, xem nhật ký của môi trường vận hành — vốn không thể và không nên đưa vào tập mặc định. Nếu cho phép cấp các quyền này một cách tự do thì tiềm ẩn rủi ro bảo mật và không thể kiểm toán; ngược lại, nếu cấp thủ công qua các kênh phi chính thức như thư điện tử hay tin nhắn thì hoàn toàn không lưu lại được dấu vết phê duyệt và không thực thi được chính sách kiểm soát.

Giải pháp đề xuất

Đề tài xây dựng một quy trình phê duyệt quyền có kiểm soát mô phỏng theo cơ chế Hội đồng tư vấn thay đổi (Change Advisory Board — CAB). Một yêu cầu cấp quyền trải qua chu trình trạng thái rõ ràng: từ bản nháp (DRAFT) chuyển thành yêu cầu chính thức (OFFICIAL) khi được gửi đi, rồi kết thúc ở một trong các trạng thái được phê duyệt (APPROVED), bị từ chối (REJECTED) hoặc bị hủy (CANCELLED). Toàn bộ thông tin được lưu trong hai bảng AUTH_REQUEST_HEADER (thông tin tổng quát của yêu cầu) và AUTH_REQUEST_DETAIL (từng mục quyền cụ thể), tạo thành một vết kiểm toán đầy đủ ghi nhận ai yêu cầu, yêu cầu cho ai, người phê duyệt là ai, vào thời điểm nào và vì lý do gì. Luồng xử lý của quy trình này đã được mô tả trong Hình ??.

Quy trình tích hợp hai cơ chế kiểm soát đáng chú ý. Thứ nhất, hệ thống bắt buộc người phê duyệt phải là thành viên có vai trò CAB và đồng thời chặn hành vi tự phê duyệt: nếu người thụ hưởng quyền trùng với người phê duyệt, yêu cầu bị từ chối ngay ở tầng nghiệp vụ. Cơ chế kiểm soát kép này được thực thi nhất quán ở cả giao diện và phía máy chủ, đảm bảo nguyên tắc phân tách trách nhiệm (*separation of duties*). Thứ hai, khi phê duyệt một quyền mà người dùng đã sở hữu một phần, hệ thống thực hiện phép hợp (union) các hành động thay vì ghi đè, tránh vô tình thu hẹp quyền hiện có.

Điểm thiết kế tinh tế nhất của quy trình nằm ở sự tương tác giữa nhãn nguồn cấp quyền và cơ chế thu hồi theo vai trò. Quyền được cấp qua phê duyệt CAB được

gắn nhãn nguồn REQUEST. Khi một vai trò của người dùng bị thu hồi, hệ thống chỉ thu hồi theo dây chuyền (cascade) những quyền có nguồn gốc từ vai trò đó, và **cố ý loại trừ** các quyền mang nhãn REQUEST. Nhờ vậy, quyền mà người dùng đã chủ động xin và được phê duyệt sẽ tồn tại độc lập với vai trò, không bị mất đi một cách bất ngờ khi cấu trúc vai trò thay đổi — một hành vi hợp lý nhưng thường bị các hệ thống RBAC đơn giản bỏ qua.

Kết quả đạt được

Quy trình cung cấp một vết kiểm toán hoàn chỉnh cho mọi quyền bổ sung, đáp ứng yêu cầu truy vết bắt buộc trong môi trường ngân hàng. Cơ chế chặn tự phê duyệt và ràng buộc vai trò CAB đảm bảo nguyên tắc phân tách trách nhiệm. Việc tách biệt nguồn cấp quyền giúp hệ thống phân biệt được quyền "thừa hưởng từ vai trò" và quyền "được cấp riêng theo yêu cầu", từ đó áp dụng chính sách thu hồi phù hợp cho từng loại.

0.2.3 Thu hồi quyền tức thì không phụ thuộc thời gian sống của token

Đặt vấn đề

Việc sử dụng JWT làm token truy cập (sẽ phân tích kỹ ở Mục 0.3) mang lại hiệu năng cao nhưng đi kèm một nhược điểm cố hữu: JWT là token tự chứa và phi trạng thái, nên một khi đã phát ra thì không thể bị thu hồi cho đến khi hết hạn. Trong tình huống nhân viên thôi việc hoặc bị thu hồi quyền, nếu token của họ vẫn còn hiệu lực thì họ vẫn có thể truy cập hệ thống cho đến hết thời gian sống của token. Với môi trường ngân hàng, kể cả một khoảng trễ vài chục phút cũng là không thể chấp nhận.

Giải pháp đề xuất

Hệ thống giải quyết mâu thuẫn giữa hiệu năng (token phi trạng thái) và an toàn (thu hồi tức thì) bằng một lớp trạng thái phía máy chủ được điều khiển qua Kafka. Khi quyền bị thu hồi hoặc nhân viên thôi việc, *iam-identity-service* phát sự kiện REVOKED-PERMISSION-NOTIFY. Bên tiêu thụ *PermissionRevokedConsumer* trong *iam-auth-service* tiếp nhận sự kiện và thực hiện hai hành động: vô hiệu hóa bản ghi phiên làm việc phía máy chủ trong bảng AUTH_CLIENT_SESSION cho cặp người dùng-ứng dụng tương ứng, và thu hồi toàn bộ refresh token của người dùng cho ứng dụng đó. Do mỗi lần làm mới token đều phải kiểm tra phiên còn hiệu lực, refresh token đã bị thu hồi sẽ không thể tạo ra token truy cập mới, buộc người dùng phải xác thực lại. Hệ thống còn xử lý tương tự cho các sự kiện liên quan: USER-CHANGED-PASSWORD thu hồi toàn bộ phiên của người dùng trên mọi ứng dụng, còn CLIENT-SECRET-RESET-NOTIFY thu hồi mọi refresh token theo định danh client.

Kết quả đạt được

Nhờ cơ chế này, hành vi thôi việc hoặc thu hồi quyền có hiệu lực trong vòng vài giây — bằng đúng thời gian lan truyền của sự kiện qua Kafka — thay vì phải chờ token hết hạn. Phạm vi thu hồi được kiểm soát ở mức từng ứng dụng: thu hồi quyền trên một ứng dụng cụ thể chỉ vô hiệu hóa phiên của người dùng trên ứng dụng đó mà không ảnh hưởng đến các ứng dụng khác mà họ vẫn còn quyền truy cập hợp lệ.

0.3 Mô hình hóa quyền trong JWT và kiểm soát truy cập phi trạng thái tại Gateway

Đặt vấn đề

Trong kiến trúc microservices, mọi yêu cầu từ người dùng đều phải được kiểm tra quyền trước khi được định tuyến đến service đích. Cách tiếp cận truyền thống là tại mỗi yêu cầu, thành phần kiểm soát truy cập sẽ truy vấn cơ sở dữ liệu để lấy quyền của người dùng rồi đối chiếu. Cách làm này khiến cơ sở dữ liệu trở thành điểm nghẽn (bottleneck) và làm tăng độ trễ của mọi yêu cầu. Một phương án khác là lưu quyền trong phiên làm việc phía máy chủ, nhưng cách này lại đòi hỏi trạng thái phiên được chia sẻ giữa các nút, làm giảm khả năng mở rộng theo chiều ngang — vốn là ưu điểm cốt lõi của microservices.

Giải pháp đề xuất

Đề tài đề xuất mô hình hóa mỗi quyền thành một chuỗi ký tự có cấu trúc và nhúng toàn bộ tập quyền của người dùng vào chính token truy cập, nhờ đó việc kiểm tra quyền trở nên hoàn toàn phi trạng thái. Một quyền được biểu diễn theo cú pháp thống nhất:

```
{serviceCode}/{resourceCode}:{action}
```

trong đó `serviceCode` định danh ứng dụng (ví dụ `iam-service`), `resourceCode` định danh tài nguyên trong ứng dụng đó (ví dụ `user`), và `action` là hành động cụ thể (ví dụ `create`). Khi đó, một quyền hoàn chỉnh có dạng `iam-service/user:create`. Cách biểu diễn dạng chuỗi này vừa cô đọng, vừa mang đầy đủ ngữ nghĩa và có thể đọc hiểu trực tiếp, thuận lợi cho việc cấu hình và kiểm toán.

Tại thời điểm phát token, *iam-auth-service* gọi một thủ tục lưu trữ (stored procedure) `AUTH_PKG.get_permission` duy nhất để lấy về toàn bộ vai trò và quyền tài nguyên của người dùng trên ứng dụng đích, sau đó kết tập (assemble) thành danh sách các chuỗi quyền và đưa vào trường `permissions` của token. Việc gom toàn bộ quyền trong một lời gọi cơ sở dữ liệu duy nhất ngay tại lúc phát

token giúp tránh hoàn toàn việc truy vấn lặp lại ở các yêu cầu sau.

Một quyết định thiết kế quan trọng là việc phân biệt hai loại token. Token cấp cho người dùng (*TokenForUser*) chứa đầy đủ vai trò và danh sách quyền, phục vụ cho việc kiểm soát truy cập tại gateway. Trong khi đó, token dùng cho giao tiếp giữa các service (*TokenForService*, cấp qua luồng *client_credentials*) chỉ chứa định danh client và phạm vi (scope), hoàn toàn không mang vai trò hay danh sách quyền. Sự tách biệt này phản ánh nguyên tắc: việc quyết định *ai được phép làm gì* được thực hiện một lần tại gateway, còn các service nội bộ chỉ cần tin tưởng rằng yêu cầu đã được gateway kiểm soát. Hình 0.3 mô tả toàn bộ luồng từ lúc phát token đến khi kiểm tra quyền tại gateway.

Giai đoạn phát token – iam-auth-service

Người dùng đăng nhập
OAuth2 + PKCE



Gọi
AUTH_PKG.get_permission
(1 lần lấy toàn bộ quyền)



Kết tập chuỗi quyền
viceCode/resourceCode:action



TokenForUser (ký ES256)
claim permissions = [iam-
service/user:create, ...]



Client gửi yêu cầu kèm
Bearer token



iam-gateway – kiểm soát phi trạng thái

JwtAuthFilter
xác minh chữ ký ES256 qua
JWKS



PermissionCheckFilter
permissions.contains(quyền
của route)?
KHÔNG truy vấn CSDL

Tại API Gateway, mỗi tuyến (route) được cấu hình ánh xạ tới đúng một chuỗi quyền yêu cầu. Bộ lọc kiểm tra quyền chỉ cần phân giải token, đọc trường `permissions` và kiểm tra xem chuỗi quyền yêu cầu của tuyến có nằm trong danh sách hay không — một thao tác đối chiếu chuỗi đơn thuần, không hề truy vấn cơ sở dữ liệu. Sau khi xác thực quyền, gateway thực hiện thêm hai bước: bơm thông tin người dùng vào các header (`X-User-Id`, `X-Employee-Code`, `X-User-Role`) để service đích sử dụng, và trao đổi token người dùng thành token dịch vụ trước khi chuyển tiếp yêu cầu. Toàn bộ token được ký bằng thuật toán ES256 và các khóa công khai được phơi bày qua endpoint JWKS, cho phép gateway và các service tự kiểm tra chữ ký token một cách độc lập mà không cần liên hệ ngược về máy chủ xác thực.

Kết quả đạt được

Nhờ mô hình này, gateway hoạt động hoàn toàn phi trạng thái: việc kiểm tra quyền chỉ gồm phân giải token và đối chiếu chuỗi, không phụ thuộc vào cơ sở dữ liệu hay bộ nhớ phiên dùng chung, nhờ đó gateway có thể mở rộng theo chiều ngang một cách tự do và đạt thông lượng cao. Đánh đổi cố hữu của hướng tiếp cận này — token cũ vẫn mang theo tập quyền cũ cho đến khi hết hạn — đã được giải quyết trọn vẹn bằng cơ chế thu hồi tức thì trình bày ở Mục 0.2.3. Sự kết hợp giữa kiểm tra quyền phi trạng thái (cho hiệu năng) và thu hồi qua sự kiện (cho an toàn) chính là điểm cân bằng mà giải pháp đạt được.

0.4 Minh chứng tích hợp ứng dụng nội bộ với nền tảng IAM

Đặt vấn đề

Một nền tảng quản lý danh tính và phân quyền chỉ thực sự chứng minh được giá trị khi có ứng dụng thực tế tích hợp và vận hành trên đó. Vì vậy, bên cạnh việc xây dựng lõi IAM, đề tài cần một minh chứng cụ thể cho thấy một ứng dụng nội bộ phát triển mới có thể sử dụng nền tảng để xác thực người dùng và phân quyền truy cập đến từng tính năng, mà không phải tự xây dựng lại các cơ chế bảo mật.

Giải pháp đề xuất

Đề tài xây dựng *demo-change-app* — một hệ thống Quản lý Thay đổi và Phát hành (Change & Go-Live Management) đóng vai trò ứng dụng nội bộ tích hợp với IAM. Ứng dụng đăng nhập thông qua luồng OAuth 2.0 Authorization Code kết hợp PKCE, sử dụng trực tiếp *iam-auth-service* làm nhà cung cấp danh tính. Về phân quyền, ứng dụng định nghĩa bộ tài nguyên và hành động riêng phù hợp với nghiệp vụ của mình — bao gồm các tài nguyên như yêu cầu thay đổi (*change-request*), phát hành (*go-live*), tài liệu (*document*), danh mục kiểm tra (*checklist*), báo cáo (*report*) và nhật ký kiểm toán (*audit-log*) — mỗi

tài nguyên đi kèm tập hành động đặc thù. Hệ thống thiết kế bốn nhóm người dùng (persona) với quyền hạn khác nhau: lập trình viên, trưởng nhóm kỹ thuật, quản lý phát hành và quản trị viên. Khi mỗi người dùng đăng nhập, token họ nhận được chứa đúng tập quyền của persona tương ứng; giao diện ứng dụng dựa trên tập quyền này để hiển thị hoặc ẩn từng tính năng, còn phía máy chủ đóng vai trò một resource server kiểm tra token độc lập.

Kết quả đạt được

demo-change-app chứng minh rằng quy trình tích hợp một ứng dụng mới vào nền tảng chỉ đòi hỏi khai báo cấu hình — đăng ký một OAuth2 client, định nghĩa tập tài nguyên—hành động, và thiết lập ma trận quyền mặc định theo vai trò—vị trí — mà không phải sửa đổi mã nguồn lõi của hệ thống IAM. Ứng dụng cũng minh họa được khả năng kiểm soát truy cập ở mức tính năng tinh tế: ví dụ lập trình viên không thể tự phê duyệt yêu cầu thay đổi do chính mình tạo ra, và chỉ quản lý phát hành mới được phép thực thi thao tác phát hành. Đây là minh chứng cụ thể cho thấy nền tảng IAM có khả năng quản lý trọn vẹn vòng đời quyền của nhiều nhóm người dùng trong một ứng dụng thực.

0.5 Kết luận chương

Chương này đã trình bày bốn đóng góp nổi bật của đề tài, xoay quanh trục phân quyền cho lập trình viên ngân hàng. Việc mở rộng RBAC theo cặp vai trò—vị trí công tác giúp biểu diễn quyền hạn ở độ mịn phù hợp mà không bùng nổ số lượng vai trò. Cơ chế quản trị tự động vòng đời quyền gắn trực tiếp việc cấp, thu hồi và tạm treo quyền với vòng đời nhân sự, đồng thời bổ sung quy trình phê duyệt có kiểm soát cho các quyền ngoài mặc định và khả năng thu hồi tức thì qua sự kiện. Việc mô hình hóa quyền thành chuỗi có cấu trúc nhúng trong JWT cho phép kiểm soát truy cập phi trạng thái tại gateway với thông lượng cao, trong khi vẫn đảm bảo an toàn nhờ kết hợp với cơ chế thu hồi qua Kafka. Cuối cùng, ứng dụng *demo-change-app* minh chứng tính khả dụng thực tế của toàn bộ nền tảng. Những kết quả đạt được, cùng các hạn chế còn tồn tại và định hướng hoàn thiện trong tương lai, sẽ được tổng kết trong Chương ??.